

Portfolio UI with Animated Expandable Cards: Research & Design Specification

Executive Summary

This report outlines a detailed design for a portfolio-style grid of cards with engaging scroll-triggered animations and clickable expansion interactions. Key requirements include a responsive card grid that animates into view (e.g. fade/slide/parallax) as the user scrolls, and where each card expands or reveals its full content on click (via a modal, inline accordion, flip effect, etc.). Transitions must be smooth, performant, and comply with accessibility best practices (keyboard navigation, ARIA roles, reduced-motion support). We recommend responsive layouts (1–4 columns at mobile/tablet/desktop breakpoints), CSS variables for design tokens (spacing, colors, shadows), and modern JS/CSS libraries such as IntersectionObserver for scroll-trigger triggers, GSAP or AOS for animations, and React-based motion libraries (Framer Motion, React Spring) for dynamic card interactions. The following sections analyze specific features, visual attributes, interaction patterns, and implementation guidance, citing official docs and design guidelines.

Key Features and Requirements

- **Card Grid Layout:** A flexible grid or list of “card” components, each containing an image/media, title, and brief summary. Cards should have consistent padding and spacing (e.g. $\geq 16\text{px}$ gaps) for clarity [12†L315-L320] [14†L343-L347] . Cards can be arranged in 1–4 columns depending on viewport (mobile: 1–2 columns; tablet: 2–3; desktop: 3–6) [14†L428-L436] .
- **Scroll-triggered Animations:** Each card should animate into view as the user scrolls (e.g. fade-in, slide-up, zoom, or subtle parallax). Animations should be smooth (typical duration $\sim 0.25\text{--}0.4\text{s}$) and triggered via a scroll observer (e.g. IntersectionObserver, GSAP’s ScrollTrigger, AOS) when cards enter the viewport [41†L55-L60] [7†L219-L222] . Use staggered timing (e.g. 100–200ms delays) for sequential cards to create a cascading effect. Subtle fade/slide effects are often preferable to dramatic spins or bounces [41†L74-L80] . Trigger thresholds (e.g. 80% of viewport) can be tuned so cards animate shortly before fully visible.
- **Clickable Expand/Reveal:** Clicking a card should reveal full details. Options include: (a) **Modal dialog** – overlay or slide-in panel (ensuring focus trap and ESC-close); (b) **Inline expansion/accordion** – the card grows in place to show more content; (c) **3D Flip** – card flips to show content on its back. All methods require smooth transitions. For modals or panels, use `role="dialog"`, `aria-modal="true"`, `aria-labelledby`, and ensure focus is trapped and restored on close [35†L36-L44] [32†L530-L532] . For inline expansion, manage `aria-expanded` on a toggle and ensure keyboard access. Clickable areas should be intuitive (whole-card click or a clear “Expand” button) [18†L189-L197] [32†L561-L566] . The close interaction can be an “X” button or clicking outside, and must be operable via keyboard (e.g. Escape key) [35†L36-L44] .
- **Smooth Transitions:** Use CSS transitions or JS animations (with easing) for opening/closing states. Recommended transition duration $\sim 0.3\text{s}$ and easing (“ease-in-out” or cubic-bezier) for expansions. For example, cards can scale up (transform: scale) or fade their contents in. Use `prefers-reduced-motion` CSS query to disable or simplify non-essential animations for users who prefer minimal motion [37†L52-L60] .

- **Accessible Keyboard Interactions:** Cards must be fully keyboard-navigable: each card or its focusable element should have `tabindex="0"` and a clear focus style (e.g. 2–3px high-contrast outline) [14†L469-L477] [19†L330-L334] . Logical tab order (left-to-right, top-to-bottom) and skip links can help with long grids [14†L474-L479] . ARIA attributes: label each card (e.g. `aria-label="Project Title"`) and use `aria-expanded` on toggles [32†L405-L413] [32†L530-L532] . For modal expansions, focus should automatically move into the modal and return to the trigger on close [35†L36-L44] .
- **Responsive Behavior:** Ensure the grid reflows on all devices. For mobile, use single-column (cards usually 100% width, touch-friendly tap targets ≥ 48 px [14†L454-L463]). For tablets/desktops, use multi-column grids with consistent gutters (≥ 16 px) [12†L315-L320] [14†L436-L444] . Images/media in cards should use fluid/ `srcset` to adapt to screen sizes.
- **Performance Considerations:** Lazy-load offscreen images and card content (native `loading="lazy"` or via IntersectionObserver) to improve speed. Define explicit image dimensions to avoid layout shifts. Preload or cache key assets for smooth scrolling (as one dev did to optimize a heavy 3D scroll effect) [1†L105-L113] . Use the `prefers-reduced-motion` media query to disable or simplify animations if the user has such a setting [37†L52-L60] . Minimize animation impact by animating `transform` / `opacity` (GPU-accelerated) only, and avoid heavy repaints. Remove expensive effects on low-power devices if needed.

Visual Attributes and Design Tokens

- **Card Size & Spacing:** Cards can be moderate size (e.g. 280–320px width on desktop, auto-fitting grid). Maintain consistent internal padding (16–24px) and external gutters (16–24px) [12†L315-L320] [14†L343-L347] . Adopt an 8px or 4pt spacing scale system for consistency [12†L315-L320] [14†L343-L347] . Define CSS variables for these (e.g. `--card-gap: 1.5rem`, `--card-padding: 1rem`).
- **Elevation and Border Radius:** Use subtle elevation: a slight box-shadow (e.g. `0 2px 8px rgba(0,0,0,0.1)`) or soft 1px border for light backgrounds [12†L315-L320] . For dark theme variants, use lighter background color to simulate depth instead of heavy shadow [19†L269-L272] . Typical corner radius is small (4–8px) for a modern look (Material Design uses 4dp). Define `--card-radius: 8px` and `--card-shadow: 0 4px 12px rgba(0,0,0,0.15)` as needed.
- **Typography Scale:** Use clear hierarchy: card titles ~18–24px on mobile (20–28px on desktop), body text ≥ 14 px [14†L399-L408] . Headings should be bold or semibold, body text regular weight. Maintain 4.5:1 contrast for primary text and CTA text (WCAG AA) [14†L362-L370] . Define scale variables (e.g. `--font-size-lg`, `--font-size-md`, etc). Line-height ~1.4–1.6 for readability.
- **Color Palette:** Choose a neutral background for cards (white or light gray) with high-contrast text [14†L360-L368] . Provide both light and dark mode variants: e.g. light theme cards on white/gray with dark text; dark theme cards on near-black/dark gray with light text. Use an accent color for highlights or CTAs (buttons/borders). Ensure hover/focus colors meet contrast guidelines [14†L372-L376] . Always pair icons with text or use ARIA labels so color alone isn't the only indicator [14†L467-L475] [14†L482-L489] .
- **Media Handling:** Card images should share a consistent aspect ratio (e.g. 4:3 or 16:9) for uniformity [14†L383-L390] . Crop or center-crop images as needed, or letterbox if variable sizes. Use responsive `srcset` and sizes, and compress images (WebP/AVIF recommended) to ~150KB/file [14†L383-L390] . Set `width` and `height` attributes to prevent Cumulative Layout Shift. Consider video or animated previews only if they loop subtly (prefer on hover, but respect reduced-motion).

Interaction Details and Animations

- **Scroll Animation Types:** Fade-in, slide-in (from bottom or sides), and scale/parallax are common. For example, `fade-in-up` (opacity 0→1, translateY -20px) or `slide-in-left`. Stagger multiple cards (e.g. 0.1s delay each) to create a “trailer” reveal. Use IntersectionObserver or GSAP/Framer triggers. AOS examples: `<div data-aos="fade-up" data-aos-duration="500" data-aos-delay="200">...</div>` [【21†L61-L69】](#) [【23†L111-L119】](#).
- **Trigger Thresholds and Anchors:** Start the animation when ~80% of card enters viewport (e.g. `threshold: 0.2`) or use anchor placement like “top-bottom” in AOS [【21†L78-L87】](#). GSAP ScrollTrigger supports precise triggers (e.g. `start: "top 80%"`). Adjust thresholds so animations feel timely.
- **Staggered Animation:** If animating a batch of cards (e.g. on initial load), use a small stagger (0.1–0.2s) for aesthetic flow. GSAP’s timeline with `.stagger` or Framer Motion’s `delayChildren` can achieve this. Example GSAP:

```
gsap.from(".card", {
  y: 20, opacity: 0, duration: 0.3, stagger: 0.15,
  scrollTrigger: { trigger: ".cards-grid", start: "top 90%" }
});
```

- **Click-to-Open Patterns:**
- **Modal/Dialog:** Clicking opens a full-screen or centered modal. Use `<div role="dialog" aria-modal="true" aria-labelledby="dialog-title">`. Trap focus inside and return focus on close [【35†L36-L44】](#). Include a visible close button. Modal animation could scale-up and fade-in.
- **Accordion/Inline Expand:** The card height grows inline to reveal content. Toggle `aria-expanded` on the card header or wrapper [【32†L530-L532】](#). Use CSS transitions (max-height or scaleY) for a smooth reveal. Ensure tab stops are managed so only expanded content is reachable.
- **Flip Card:** Using CSS 3D transforms, flip on the Y-axis to show the back side. Ensure the back side has its own content and a visible close (or click again). Respect reduced-motion (avoid flip for motion-sensitive users [【37†L52-L60】](#)).
- **Close Interactions:** The user must easily close expanded content: clicking outside the modal or a clear [X] button, and pressing “Escape”. For accordion, a second click on the card can collapse it. After closing a modal, return focus to the card that opened it [【35†L36-L44】](#).
- **Focus Management:** If using a modal, use a focus trap: all Tab/Shift+Tab keys should cycle within the modal [【35†L36-L44】](#). The close button should be first or last in tab order. For inline expansion, ensure focus moves into the new content (or at least the card remains focus). Visible focus outline is mandatory (do not `outline:none`; use a custom 3px outline if needed for brand) [【19†L330-L334】](#).
- **ARIA Attributes:**
- For cards as controls: add `tabindex="0"` and `role="button"` if the whole card is clickable. Use `aria-label` to name the card or nest a heading inside a focusable element [【32†L405-L413】](#).
- If the card expands inline, include `aria-expanded` on the trigger element (set to true/false) and `aria-controls` linking to the expandable content.
- For modal toggles, wrap in a `<button>` or give appropriate `aria-` attributes. On the modal dialog element, use `role="dialog"`, `aria-modal="true"`, `aria-labelledby` (the modal title), and optionally `aria-describedby` [【35†L36-L44】](#).

- Ensure any icons/buttons have `aria-label` (e.g. a close icon button should have `aria-label="Close"`).

Implementation Guidance and Libraries

- **Intersection Observer (Web API):** Use this native API for scroll-trigger logic. It asynchronously watches elements entering the viewport, avoiding manual scroll handlers [8†L209-L212] [8†L257-L261]. For example:

```
const cards = document.querySelectorAll('.card');
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.classList.add('visible'); // trigger CSS animation
      observer.unobserve(entry.target);
    }
  });
}, { threshold: 0.2 });
cards.forEach(card => observer.observe(card));
```

The MDN docs note that IntersectionObserver frees the main thread for performance and enables lazy-loading use cases [8†L230-L233] [8†L257-L261].

- **Animation Libraries:**

- **GSAP (GreenSock) with ScrollTrigger:** Powerful for complex, choreographed scroll animations. Its `ScrollTrigger` plugin can pin, scrub, and link animations to scroll position [7†L219-L222] [7†L289-L293]. GSAP uses hardware-accelerated transforms for performance. Example:

```
gsap.registerPlugin(ScrollTrigger);
gsap.from(".card", {
  opacity: 0, y: 30, duration: 0.4,
  scrollTrigger: { trigger: ".card", start: "top 80%", toggleActions:
    "play none none none" }
});
```

- **Framer Motion (React):** A React library with layout animations out-of-the-box. Adding the `layout` prop on a `<motion.div>` animates between layout changes automatically [25†L91-L94]. It performs animations via CSS transforms for efficiency [25†L155-L159]. Use `AnimatePresence` for mounting/unmounting (modals) and `layoutId` for shared element transitions between cards and expanded views [25†L193-L199].
- **React Spring:** A physics-based animation library for React. Use hooks like `useSpring` or `useTransition` for animations (e.g. spring-open a card). It's lightweight and declarative, though a bit more low-level than Framer Motion. (No single citation here, but use official docs examples.)
- **AOS (Animate On Scroll):** Simple vanilla library that adds animation classes via data attributes. E.g. `<div data-aos="fade-up" data-aos-duration="500"></div>` [21†L61-L69] [23†L111-L119]. It has no dependencies, which keeps bundle size small [23†L159-L161]. Use it for quick prototypes or static sites; configure with `AOS.init()`.

- **CSS Animations & Libraries:** If minimal JS is desired, simple CSS keyframe animations with `animation-play-state: paused` (toggled on intersection) can suffice [3†L74-L82] [4†L74-L80]. Libraries like Animate.css or AOS just add CSS classes.
- **Code Snippets:**
- *HTML/CSS Example for a Card:*

```
<article class="card" tabindex="0" role="button" aria-expanded="false"
aria-label="Project Alpha">
  
  <h3>Project Alpha</h3>
  <p>Short summary of the project...</p>
</article>
```

```
.card { background: var(--card-bg, white); border-radius: var(--card-
radius); box-shadow: var(--card-shadow); padding: 1rem; transition:
transform 0.3s ease; }
.card:focus { outline: 3px solid var(--focus-color); }
.card.visible { opacity: 1; transform:
translateY(0); } /* initial .card { opacity:0; transform:
translateY(20px);} */
```

- *React Component Sketch:*

```
function PortfolioCard({ title, image, summary, details }) {
  const [open, setOpen] = useState(false);
  return (
    <article className="card" tabIndex={0}
      role="button" aria-expanded={open}
      onClick={() => setOpen(true)}>
      <img src={image} alt={title} />
      <h3>{title}</h3>
      <p>{summary}</p>
      {open && (
        <Modal onClose={() => setOpen(false)} aria-label={title}>
          <h2>{title}</h2>
          <p>{details}</p>
          <button onClick={() => setOpen(false)}>Close</button>
        </Modal>
      )}
    </article>
  );
}
```

This shows a card that opens a `<Modal>` (with focus trap & `role="dialog"`) when clicked. Adjust the markup for inline expansion similarly.

- *IntersectionObserver Setup:*

```
const observer = new IntersectionObserver(callback, { threshold:
0.15 });
function callback(entries) {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.classList.add('animate-in');
      observer.unobserve(entry.target);
    }
  });
}
document.querySelectorAll('.card').forEach(card =>
observer.observe(card));
```

- **ARIA & Screen Reader Notes:** Use semantic elements (`<article>`, `<button>`, `<img alt>`, headings) where possible. For purely decorative images, use empty alt. Use ARIA labels on icons/ buttons [32†L405-L413] . Ensure the expand control has `aria-expanded` and an accessible name. Announce any dynamic content changes (e.g. “Project X details opened”).

Accessibility and Performance Checklist

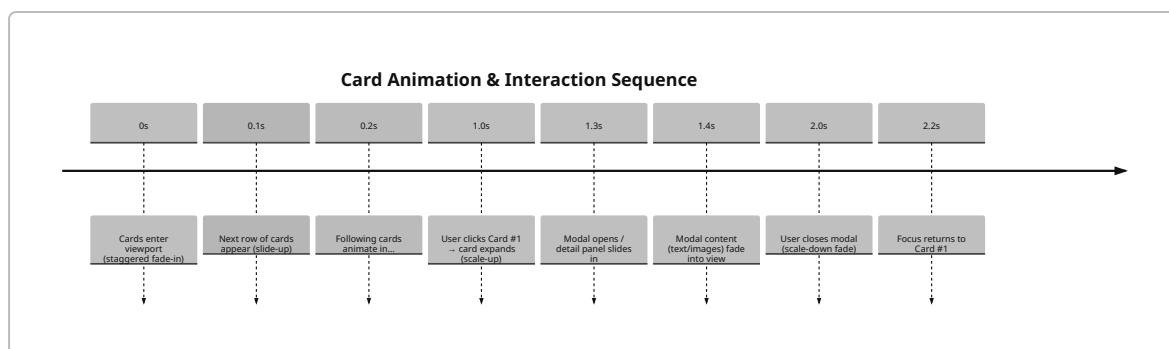
- **Keyboard:** All interactive elements reachable with Tab; `tabindex="0"` on card containers if needed. Focus outlines visible [14†L469-L477] . Logical tab order. Escape key closes modals [35†L36-L44] .
- **ARIA:** Cards have accessible names (`aria-label` or `<h3>` inside a `<button>` / `<a>`). Expand controls have `aria-expanded` . Modals use `role="dialog"` , `aria-modal="true"` , labelled by a heading [35†L36-L44] . Toggle buttons include `aria-label` if icon-only [32†L514-L522] .
- **Contrast & Colors:** Text meets WCAG AA (4.5:1) [14†L366-L370] . Never rely on color alone (use icons or text) [14†L503-L508] . Provide dark mode with adjusted backgrounds and shadows [14†L501-L508] [19†L269-L272] .
- **Reduced Motion:** Respect `prefers-reduced-motion` ; disable non-critical animations or use simpler fade-ins [37†L52-L60] .
- **Performance:** Lazy-load offscreen images (`loading="lazy"` or observer) [14†L385-L390] [8†L230-L233] . Optimize images (WebP/AVIF, small file size) and use `srcset` . Batch animations efficiently (e.g. GSAP's `batch` or Framer's `useScroll`). Profile on mobile to avoid jank.

Animation Libraries Comparison

Library / Tool	Type	Key Features	Use Cases & Notes
IntersectionObserver	Native Web API	Asynchronous viewport detection; no extra weight [8†L257-L261] ; lazy-load support	Trigger CSS/JS animations when in view; great for performance-sensitive scroll events.

Library / Tool	Type	Key Features	Use Cases & Notes
AOS (Animate On Scroll)	Vanilla JS/ CSS	Simple data-attribute API (data-aos); many presets (fade, slide, flip) [23†L111-L119] ; no dependencies [23†L159-L161]	Quick scroll animations without manual code. Ideal for static sites or minimal JS.
GSAP (ScrollTrigger)	JS Animation Library	Powerful timeline control; scrub/pin/snap; supports complex sequences [7†L219-L222] ; high performance (uses transforms)	Any scroll-based effect (parallax, pinning, scrubbing). Suitable for complex, custom animations.
Framer Motion	React Animation Library	Declarative React API; layout animations with <code>layout</code> prop [25†L91-L94] ; <code>AnimatePresence</code> for modals; hardware-accelerated transforms [25†L155-L159]	React apps. Excellent for animating layout changes and shared element transitions (e.g. smooth card-to-modal).
React Spring	React Spring Physics	Spring-based physics animations; hooks API (<code>useSpring</code> , <code>useTransition</code>)	React; fine-grained, physics-driven animations (e.g. spring-open accordions, dragging). Lightweight for interactive UIs.

Animation Sequence Timeline



Deliverables: A final design spec should include a component list, CSS variables (e.g. `--card-gap`, `--card-radius`, typography scale, color palette variables for light/dark), and animation timing tokens (e.g. `--anim-duration: 0.3s`, `--anim-easing: ease-in-out`). Supply a working example code snippet for one card (HTML/JSX + CSS) and its expanded view, demonstrating key behaviors (ARIA attributes, animation triggers).

Design Prompt: Create a responsive portfolio layout with a grid of cards, each animating into view on scroll (e.g. fading or sliding in). Cards should have a consistent size, subtle shadow, and rounded corners. When a card is clicked or activated via keyboard, it should smoothly expand or open a modal to show full details.

Include smooth CSS transitions (around 300ms) and JavaScript for interactive behavior. The UI must be fully accessible: cards are focusable (with visible outline), keyboard users can open/close the card details, and focus is managed in any modal. Use ARIA roles (e.g. `role="dialog"`, `aria-expanded`) appropriately. Ensure responsive breakpoints (1 column on mobile, 3–4 on desktop) and performance optimizations (lazy-load offscreen images, respect `prefers-reduced-motion`). Supply a design spec with CSS variables for spacing/shadows/colors, a component inventory, and sample code for one card and its expanded detail view. This spec will guide the developer or AI in recreating the portfolio UI described.
